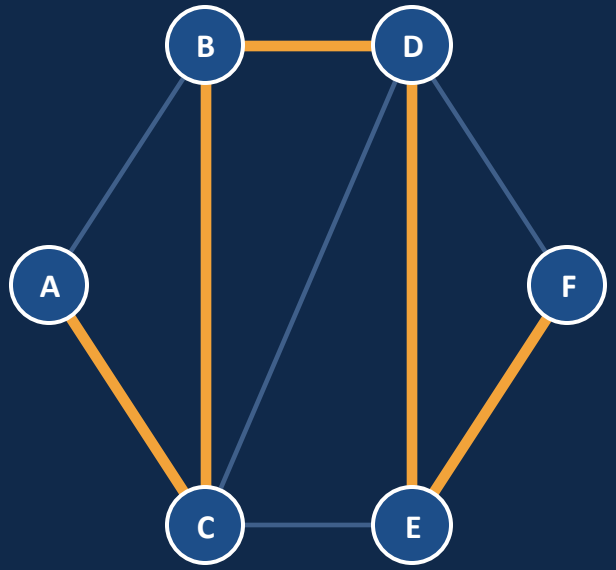
Cloud Task Placement using MST, Shortest Paths and Dynamic Programming

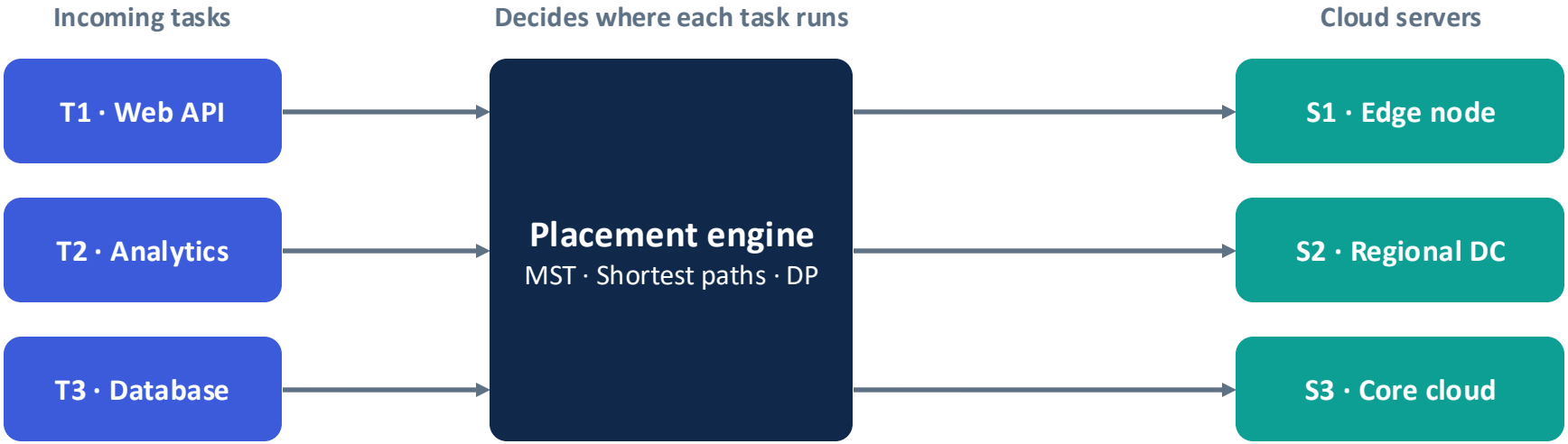
A graph-algorithmic approach to cost-efficient workload scheduling



Example network of six servers; the highlighted links form a minimum spanning tree

The Cloud Task Placement Problem

Every task must run on some server, and that choice drives cost, latency and load.



1 **Minimize total cost**

2 **Cut latency**

3 **Balance the load**

Modeling the Cloud as a Weighted Graph

1 **Servers become nodes**
Each data center or VM host has a capacity and an execution price.

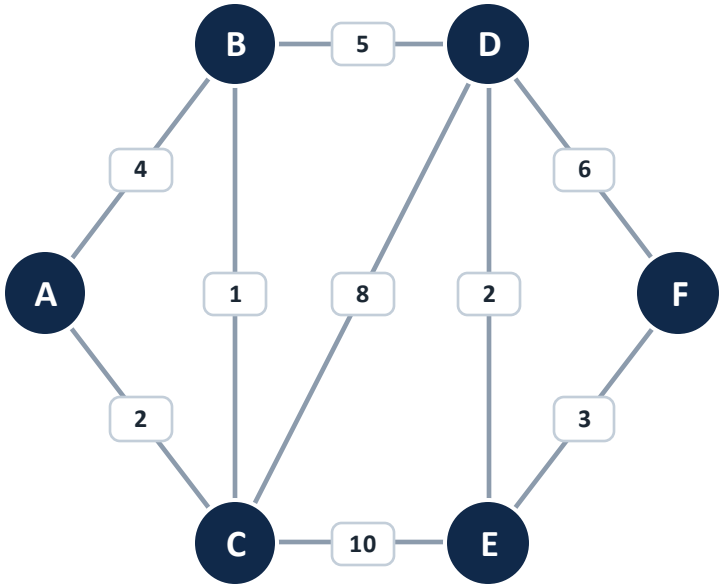
2 **Links become weighted edges**
The weight is the latency or transfer cost between two servers (ms).

3 **Tasks become demands**
Each task needs CPU and memory and exchanges data with other tasks.

Goal: choose a server s(i) for every task i to minimize

$$\Sigma\, e(i, s(i)) \;+\; \Sigma\, c(s(i), s(j))$$

execution cost + communication cost between dependent tasks



Example network: 6 servers (A–F), link latency in ms

Three Tools, Three Roles

1 STRUCTURE

Minimum Spanning Tree

Connects all servers with the cheapest set of links and reveals natural clusters.

Output: clusters and backbone

2 DISTANCE

Shortest Paths

Finds the cheapest route between any two servers, giving the communication cost c(k, j).

Output: cost matrix

3 OPTIMIZATION

Dynamic Programming

Builds the best task-to-server assignment from optimal answers to smaller sub-problems.

Output: placement plan

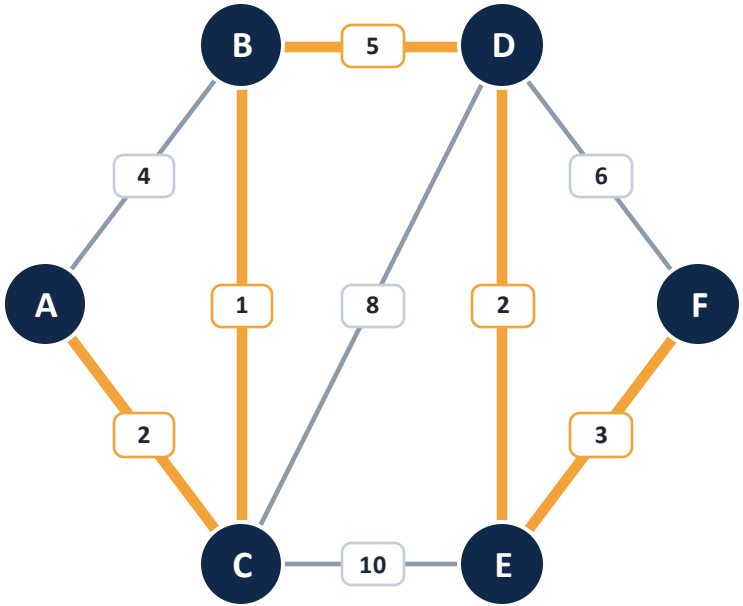
Build the topology → Measure the costs → Optimize the assignment

MST: Cheapest Way to Connect Servers

Kruskal: sort edges by weight and keep each edge that does not close a cycle.

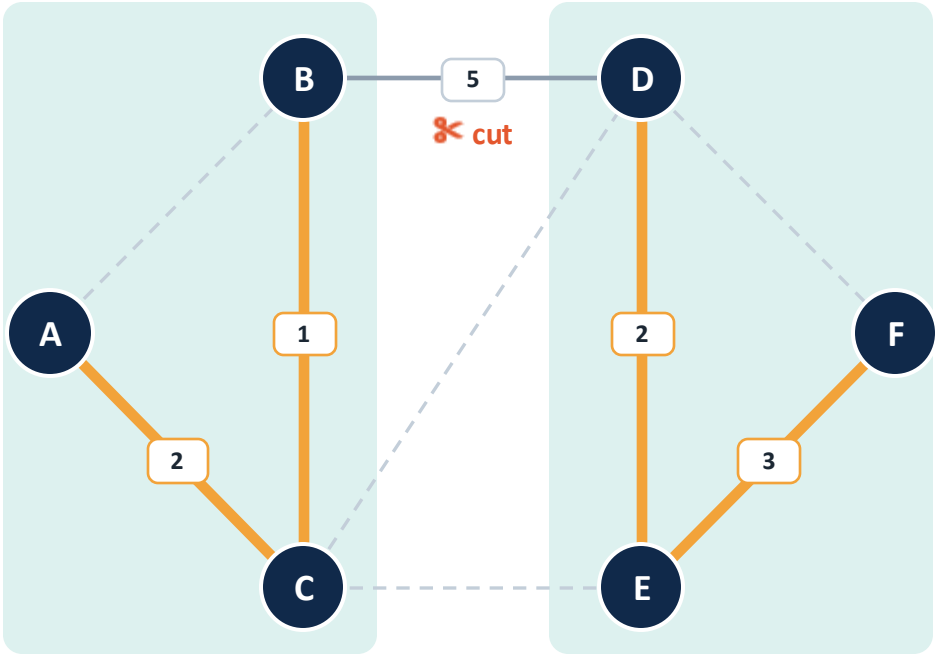
Edge	Weight	Decision
B – C	1	Add
A – C	2	Add
D – E	2	Add
E – F	3	Add
A – B	4	Skip (cycle)
B – D	5	Add

MST cost = 1 + 2 + 2 + 3 + 5 = 13 ms using V – 1 = 5 links



Amber links form the MST; the grey edges are dropped

Using the MST to Cluster and Place Tasks

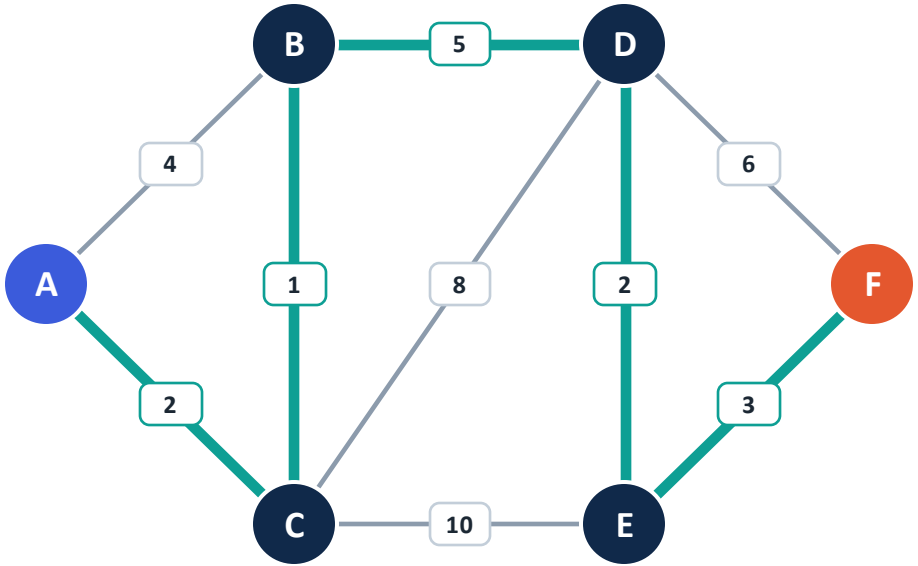


1 **Cluster by cutting**
Remove the heaviest MST edge (B–D, 5 ms) to get two zones with tight internal latency.

2 **Co-locate chatty tasks**
Tasks that exchange lots of data share a zone, avoiding the costly inter-zone link.

3 **Prune the search**
The DP step first compares servers inside a zone, which shrinks m in $O(n \cdot m^2)$.

Shortest Paths: Pricing Communication



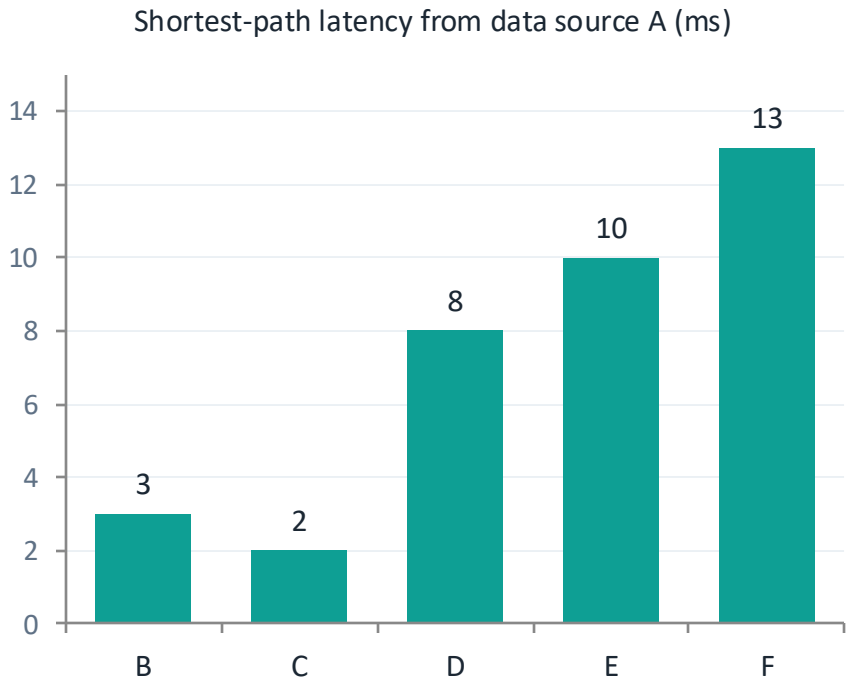
Shortest A → F: A–C–B–D–E–F = 13 ms
(A–B–D–F looks shorter but costs 15 ms)

Dijkstra from source A

Node	Distance	Previous
A	0	–
C	2	A
B	3	C
D	8	B
E	10	D
F	**13**	**E**

Relaxation rule
$d(v) = \min(d(v), d(u) + w(u, v))$

Shortest Paths Guide Task Placement



1 Data affinity
Place data-heavy tasks on nodes with small d(source, node), such as B and C.

2 Latency SLA
With an 8 ms limit, only A, B, C and D are feasible hosts.

3 Cost matrix
Run Dijkstra from every node to build c(k, j) for the DP step.

Dynamic Programming: Optimal Assignment

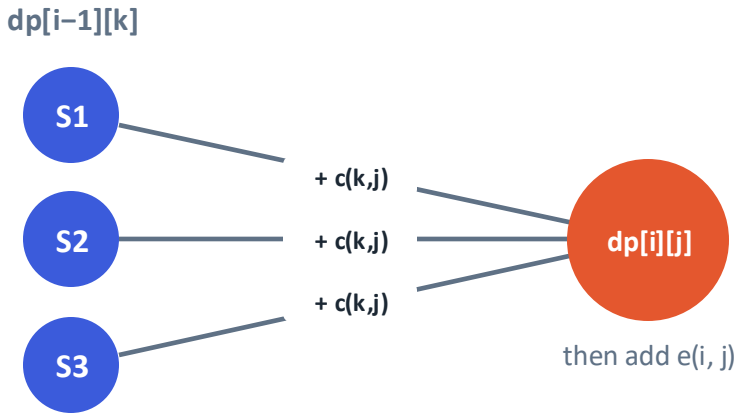
1. **State and base case**
   dp[i][j] = best cost of tasks 1..i with task i on server j;
   dp[1][j] = e(1, j).

2. **Optimal substructure**
   An optimal plan for i tasks contains an optimal plan for i − 1 tasks, so sub-results are reused.

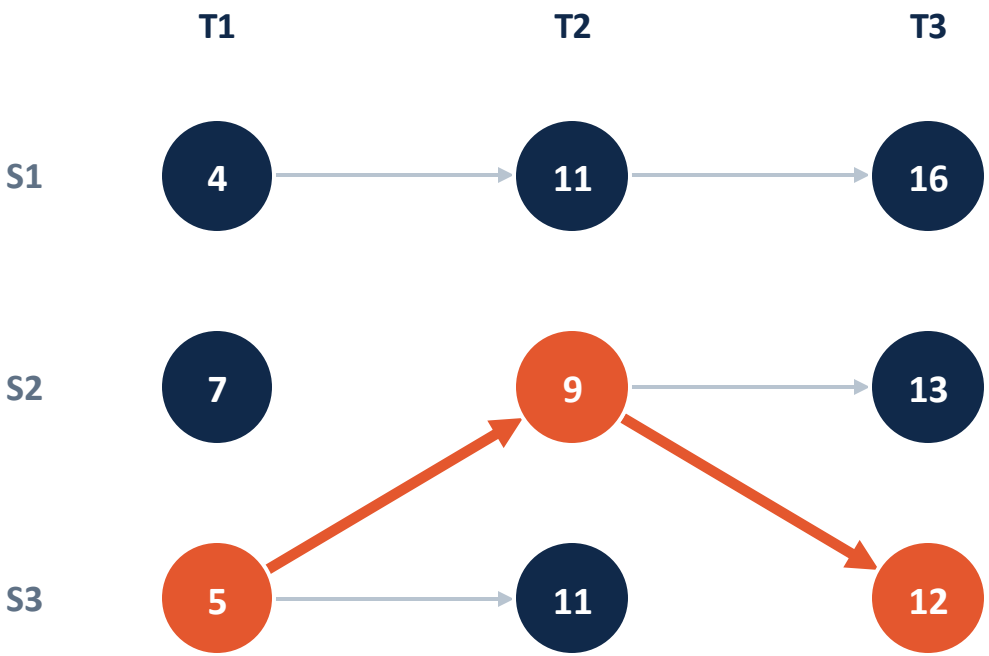
3. **Fill the table**
   Apply the recurrence for each task and server, storing the best predecessor k.

4. **Backtrack**
   Take the minimum at the last task and follow the pointers to recover the placement.

dp[i][j] = e(i, j) +
min over k of (dp[i−1][k] + c(k, j))
e = execution cost · c = shortest-path cost between servers (0 if same)



DP Worked Example: Three-Task Pipeline



Node = least cost to finish task i on that server. Coral path = optimal.

Inputs: execution cost e(i, j)

	S1	S2	S3
T1	4	7	5
T2	7	3	6
T3	5	4	2

Link cost c: S1–S2 = 3, S2–S3 = 1, S1–S3 = 4; same server = 0

Result

DP optimum: T1→S3, T2→S2, T3→S3 = 12

Greedy (cheapest server per task): 13

Everything on S3: 13

Integrated Placement Framework



Build the graph of servers and link latencies.

Find the backbone and cut it into zones.

Compute the cost matrix c(k, j).

Assign tasks to servers at minimum total cost.

Launch tasks on the chosen servers.

Feedback loop
Monitor load and latency, then re-run steps 2–4 whenever the topology or the demand changes.

Comparing the Three Techniques

Technique	Question it answers	Time	Strength	Limitation
MST (Kruskal, Prim)	Which links connect all servers most cheaply?	O(E log V)	Fast clustering and a cheap backbone	Tree paths are not shortest paths between pairs
Shortest paths (Dijkstra)	What is the cheapest route between two servers?	O((V+E) log V) per source	Exact communication costs	Needs non-negative weights
Dynamic programming	Which server should each task use?	$O(n \cdot m^2)$	Exact optimum for task chains	State grows with capacity limits and general DAGs

Together: MST shapes the topology, shortest paths price it, DP optimizes the assignment.

Conclusion and Future Scope

Key takeaways

1. Cloud placement is a graph problem with cost, latency and load goals.
2. MST gives clusters, shortest paths give costs, DP gives the optimal plan.
3. In the example, DP cut the cost from 13 to 12 versus greedy placement.

Future scope

- Capacity and multi-resource constraints (CPU, memory, bandwidth)
- Online placement as tasks arrive and leave
- Learned latency and cost predictions
- Energy-aware and carbon-aware placement